

Desarrollo de software para el cálculo de parámetros acústicos ISO 3382

Lautaro Castrillo, Salvador Pellegrino, Ivan Maiolo

Señales y Sistemas, Ingeniería de Sonido

Universidad Nacional de Tres de Febrero, Buenos Aires, Argentina

Resumen

El presente trabajo detalla el diseño, desarrollo e implementación de una interfaz de programación de aplicaciones (API) en Python orientada al cálculo de parámetros acústicos (EDT, T10, T20, T30, C80 y D50) en estricta conformidad con las normativas ISO 3382 e IEC 61260. El sistema integra la síntesis de señales de excitación —como ruido rosa y sine sweep logarítmico— para la obtención de la respuesta al impulso. El procesamiento posterior abarca el cálculo de la envolvente mediante la Transformada de Hilbert, el análisis del decaimiento energético vía integración inversa (optimizada con el algoritmo de Lundebay) y el ajuste de resultados por regresión lineal. Finalmente, la arquitectura matemática fue expuesta a través de endpoints web, logrando aislar el motor de cálculo y demostrando una gran robustez frente a software de referencia en la industria como Room EQ Wizard (REW).

1. Introducción

Este proyecto tiene como objetivo desarrollar un software para calcular parámetros acústicos bajo los lineamientos de la norma ISO 3382 [1]. Esta normativa define los métodos para medir el tiempo de reverberación en recintos mediante respuestas al impulso, las cuales pueden generarse por síntesis o mediante la captura de barridos de frecuencia (sine sweep). Para garantizar la escalabilidad del sistema, la implementación se consolidó como una Interfaz de Programación de Aplicaciones (API). Esta arquitectura aísla el motor de procesamiento matemático de la capa web, proporcionando la versatilidad necesaria para utilizar el núcleo del software sin depender de una interfaz gráfica tradicional.

2. Marco Teórico

2.1. Ruido Rosa

El ruido rosa es un ruido cuyo nivel en función de la frecuencia cae 3 dB por octava. Su densidad espectral de potencia (PSD) es proporcional a la inversa de la frecuencia ($1/f$). Para estimar y validar correctamente la densidad espectral en la fase de diseño, se optó por utilizar el algoritmo de Voss-McCartney [2]. Este método divide la señal en ventanas superpuestas, calcula el periodograma mediante la Transformada Rápida de Fourier (FFT) de cada segmento y los promedia, reduciendo así drásticamente la varianza del espectro resultante frente a un periodograma estándar. Esto garantiza un nivel de energía constante por bandas de octava, reflejando de manera precisa la audición de tipo logarítmico del oído humano. En la fig. (1) puede observarse la densidad de potencia del ruido rosa en función de la frecuencia.

2.2. Sine Sweep Logarítmico y Filtro Inverso

Para excitar un recinto evitando las distorsiones de fuentes impulsivas, se utiliza un barrido de frecuencias (sine sweep logarítmico). La señal de excitación en el

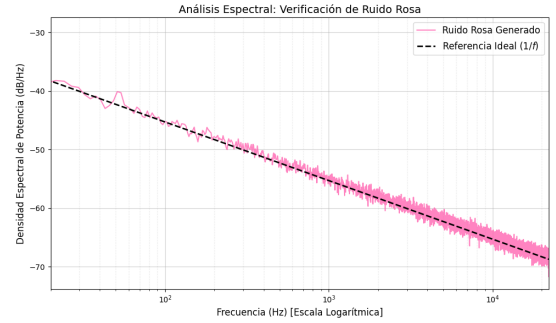


Figura 1: Espectro del ruido rosa sintetizado.

dominio temporal está dada por:

$$f(t) = \sin[K(e^{t/L} - 1)] \quad (1)$$

Donde T es el tiempo de duración, el *sweep rate* es $R = \ln(\frac{\omega_2}{\omega_1})$, $K = \frac{T\omega_1}{R}$ y $L = \frac{T}{R}$, siendo ω_1 y ω_2 las frecuencias angulares inferior y superior.

Para extraer la Respuesta al Impulso (RI) de la sala, se implementó la técnica de Farina (2000) [3]. Asumiendo que la sala es un sistema Lineal e Invariante en el Tiempo (LTI), queda caracterizada por su RI $h(t)$. Al reproducir el barrido $x(t)$, la grabación resultante $y(t)$ es la convolución $y(t) = x(t) * h(t)$.

El objetivo ideal sería inyectar un impulso perfecto o delta de Dirac $\delta(t)$, ya que $\delta(t) * h(t) = h(t)$, pero físicamente es inviable debido a las limitaciones del parlante. El método de Farina propone una solución ingeniosa: utilizar el barrido $x(t)$ y su filtro inverso $x_{inv}(t)$, los cuales, al convolucionarse, generan un impulso perfecto ($x(t) * x_{inv}(t) \approx \delta(t)$). Aplicando la propiedad asociativa de la convolución, al procesar la grabación con el filtro inverso se obtiene la RI de la sala:

$$\begin{aligned} y(t) * x_{inv}(t) &= [x(t) * h(t)] * x_{inv}(t) \\ &= h(t) * [x(t) * x_{inv}(t)] \\ &\approx h(t) * \delta(t) = h(t) \end{aligned} \quad (2)$$

A nivel algorítmico, esta deconvolución se ejecuta de forma altamente eficiente en el dominio de la frecuencia

mediante la Transformada Rápida de Fourier (FFT), tomando como herramienta a la multiplicación espectral, que equivale a la convolución temporal. La decisión de diseño de utilizar `fftconvolve` acelera el procesamiento varios órdenes de magnitud frente a una convolución temporal pura.

En la fig. (2) pueden observarse las señales mencionadas, así como su convolución.

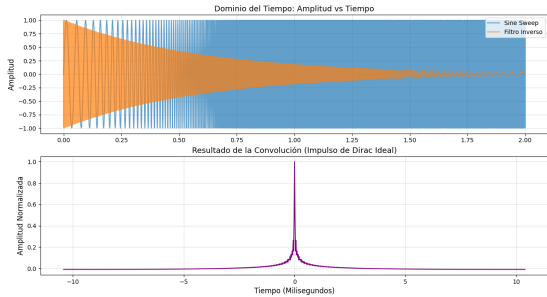


Figura 2: Sine sweep logarítmico, su filtro inverso y la convolución entre ambos.

2.3. Respuesta al Impulso

La respuesta al impulso de un sistema es la huella digital que lo identifica acústicamente. Si bien la API permite generar una RI de manera artificial (sintetizada) con fines de calibración y control, el método principal en campo consiste en excitar el recinto in situ mediante el *sine sweep*. Al grabar la respuesta de la sala y aplicarle la deconvolución de Farina mencionada en la sección anterior, la API extrae, normaliza y exporta la verdadera respuesta al impulso del sistema. En la Figura 3 puede observarse el decaimiento de la respuesta al impulso recuperada exclusivamente mediante este método.

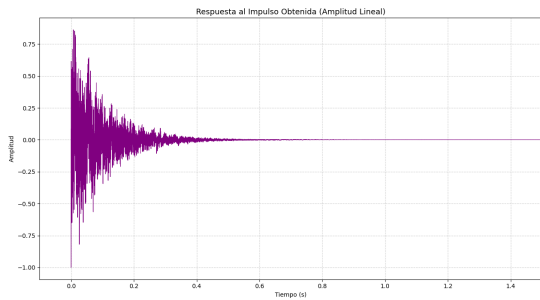


Figura 3: Respuesta al Impulso obtenida a partir de la deconvolución del sine sweep.

2.4. Filtros de Octava

Para calcular los parámetros acústicos y evaluar las diferencias en un mismo parámetro en función de la frecuencia, se llevó a cabo una función que filtra la señal en bandas de octava normalizadas según la norma IEC 61260 [4]. En la fig. (4) se expone un banco de filtros, donde se tomó un impulso y se lo filtró en cada una de las bandas de octava, para verificar el correcto funcionamiento del filtro. En la fig. (5) se expone el funcio-

namiento del filtro de octava centrado en 1 kHz para el espectro de potencia de un ruido rosa.

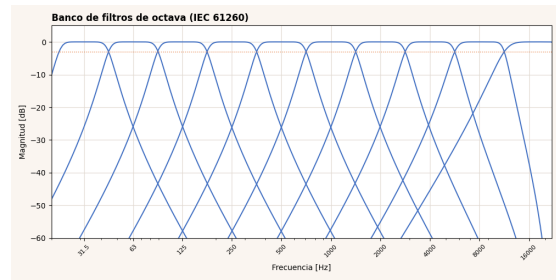


Figura 4: Banco de filtros pasabanda.

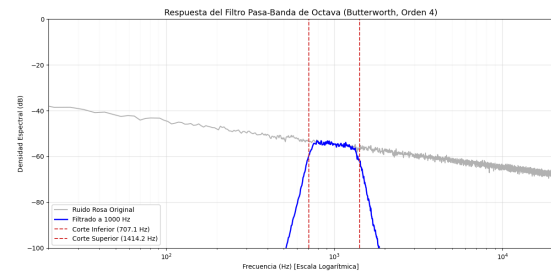


Figura 5: Filtro de banda de octava individual aplicado a la señal.

2.5. Conversión a Escala Logarítmica

Convertir la amplitud lineal de la respuesta al impulso a una escala logarítmica de decibels (dB) es un paso matemático y psicoacústico indispensable. En el dominio lineal, la inmensa diferencia de energía entre el arribo del sonido directo y la cola reverberante hace que la caída se vea aplastada contra el cero, ocultando el verdadero comportamiento acústico. Al aplicar la transformación logarítmica y normalizar el pico máximo a 0 dB, se logra visualizar la caída de forma proporcional a cómo el oído humano percibe la atenuación sonora. Esta operación convierte un decaimiento visualmente confuso en una pendiente recta lista para ser procesada. En la fig. (6) se expone una respuesta al impulso sintetizada, en amplitud lineal como en escala logarítmica.

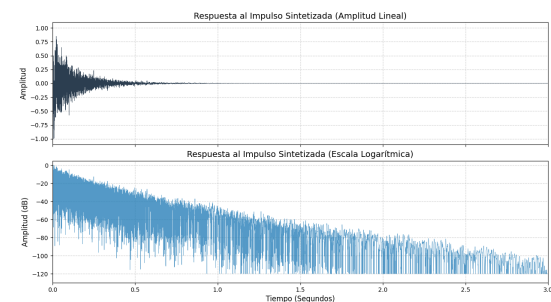


Figura 6: Respuesta al Impulso en amplitud lineal vs escala logarítmica.

2.6. Suavizado: Transformada de Hilbert y Media Móvil

El análisis de una RI resulta complejo debido a las oscilaciones propias de la señal. Para estudiar parámetros acústicos de interés, es necesario analizar el decaimiento de la energía a través del tiempo. En este contexto, resulta indispensable el cálculo de la envolvente energética. Por eso, se implementó la función de suavizado, que brinda una opción doble. El suavizado por Media Móvil se define mediante la siguiente expresión:

$$y[n] = \frac{1}{M} \sum_{k=0}^{M-1} x^2[n-k] \quad (3)$$

Donde $x^2[n]$ representa la energía instantánea de la señal. La elevación cuadrado es necesaria para trabajar en términos energéticos. El término M representa el tamaño de una ventana. Muchas veces, escoger el tamaño de dicho parámetro resulta complejo, por eso se prefiere el suavizado mediante la transformada de Hilbert para extraer su envolvente y suavizarla, permitiendo analizar el decaimiento dinámico de la energía. La transformada de Hilbert $H(x)(t)$ de una señal real $x(t)$ se define como:

$$H(x)(t) = \frac{1}{\pi} \int_{-\infty}^{\infty} \frac{x(\tau)}{t - \tau} d\tau \quad (4)$$

A partir de ella se forma la señal analítica $z(t) = x(t) + jH(x)(t)$. La magnitud de esta señal, $|z(t)| = \sqrt{x^2(t) + H(x)^2(t)}$, conforma la envolvente temporal exacta de la respuesta acústica. Posteriormente, esta envolvente se convierte a escala logarítmica (dB). En la fig. (7) se observa la curva energética obtenida por media móvil, y la curva obtenida por Hilbert.

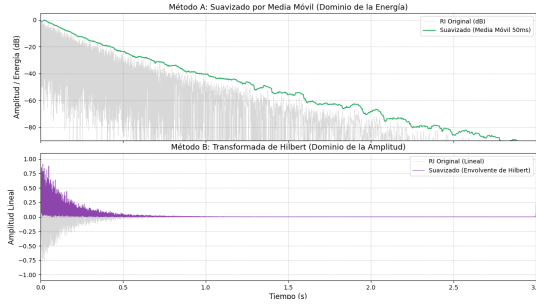


Figura 7: Suavizado de la señal mediante Media Móvil (arriba), y mediante transformada de Hilbert (abajo).

2.7. Cálculo de Parámetros Acústicos

Para extraer los parámetros acústicos, primero es fundamental suprimir la varianza estadística del ruido. Para ello se aplica la integral de Schroeder [5], que transforma la respuesta al impulso en una curva de decaimiento de energía estrictamente monótona decreciente, calculada discretamente como una sumatoria invertida:

$$EDC[n] = \sum_{i=n}^N h^2[i] \quad (5)$$

Para evitar que el piso de ruido ambiental distorsione la integral, se utiliza el método de Lundebly [6]. Este

estima el punto de cruce exacto entre la energía reverberante y el ruido, estableciendo una cota temporal segura. Sobre esta curva integrada y truncada (ver Figura 8), se aplican regresiones lineales por mínimos cuadrados para estimar las pendientes de decaimiento (Figura 9).

A partir de estas rectas, se extrapola matemáticamente el tiempo que tardaría la energía en decaer 60 dB. Los parámetros principales (T20 y T30) se evalúan en sus respectivos rangos de caída y se multiplican por su factor correspondiente:

$$T_{20} = 3 \times (t_{-25\text{dB}} - t_{-5\text{dB}}), \quad T_{30} = 2 \times (t_{-35\text{dB}} - t_{-5\text{dB}}) \quad (6)$$

El EDT evalúa el decaimiento temprano (0 dB a -10 dB) multiplicándose por 6. De esta manera, el cálculo de parámetros resulta de una concatenación estricta entre Schroeder, Lundebly y regresión lineal.

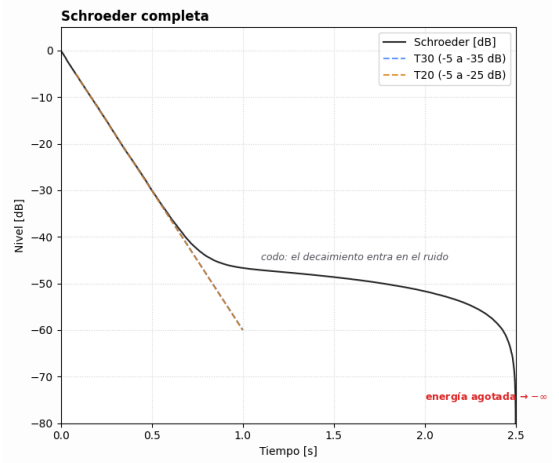


Figura 8: Early Decay Curve obtenida mediante la integral de Schroeder.

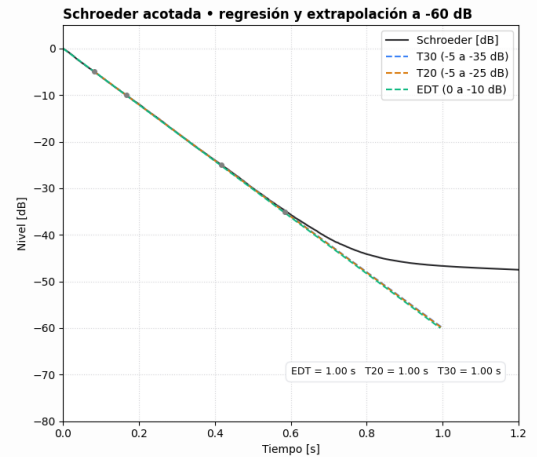


Figura 9: Ajuste de regresión lineal para la obtención de T20 y T30, utilizando el método Lundebly.

3. Validación y Testing

Para asegurar la calidad de la API, se implementó una estrategia robusta de pruebas automáticas mediante el

framework *Pytest*. Como decisión arquitectónica clave, se inyectaron rutas temporales (`tmpdir`) para procesar audios y gráficos exclusivamente en la memoria volátil, acelerando exponencialmente la validación sin dejar datos residuales.

La batería de pruebas se fundamentó en los siguientes criterios estratégicos:

- Casos ideales y extremos para probar los límites matemáticos.
- Inyección de ruido intencional en las señales.
- Verificación de la resiliencia del algoritmo de Lundebly.
- Regresiones lineales evaluadas bajo condiciones de acústica "sucias".
- Altísimos niveles de precisión con bajas tolerancias de error.

El motor superó con éxito el 100 % de las operaciones (Figura 10).

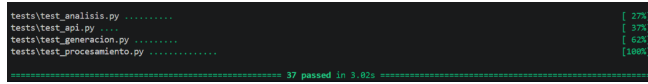


Figura 10: Resultados exitosos de la batería de tests automatizada.

4. Análisis de Resultados

Se evaluó la eficacia del sistema midiendo respuestas al impulso en salas de diferentes características, contrastando el desempeño frente a Room EQ Wizard (REW).

4.1. Comparativa Paramétrica: API vs REW

Al comparar los parámetros obtenidos por la API y por el software REW para la RI sintetizada y el recinto "Theatre", se aprecia una altísima convergencia en las bandas centrales de mayor interés acústico (500 Hz, 1 kHz y 2 kHz).

La utilización de una Respuesta al Impulso Sintetizada funciona como un *ground truth* o patrón de control absoluto. Al estar libre de ruido acústico real, permite auditar exclusivamente la precisión matemática del código frente a REW. Como se observa en la Figura 11 y el Cuadro 1, la correlación en T20 y T30 es casi idéntica, garantizando que el motor de cálculo y los filtros Butterworth de fase cero (implementados mediante `filtfilt`) no introducen distorsiones temporales artificiales.

Al pasar al recinto empírico "Theatre", se evalúa la herramienta frente a una grabación de sala con decaimiento limpio y bien comportado. La Figura 12 y el Cuadro 2 exponen que las discrepancias se mantienen al mínimo en las bandas centrales. Pequeñas variaciones en el EDT pueden atribuirse a las diferencias algorítmicas internas en cómo cada software (API vs REW) maneja el suavizado de las primeras reflexiones.

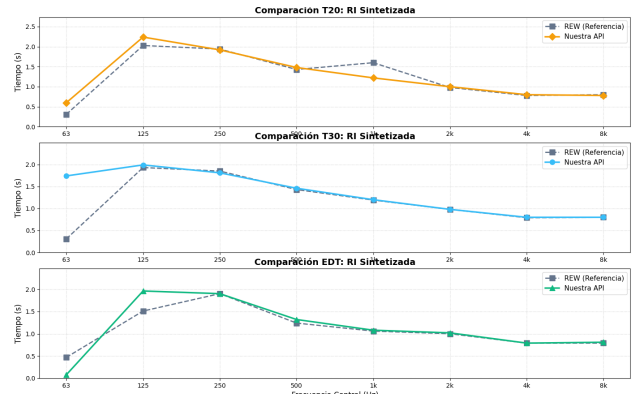


Figura 11: Comparativa paramétrica frente a REW (RI Sintetizada).

Parámetro	Software	500 Hz	1 kHz	2 kHz
T20	REW	1.43	1.6	0.98
T20	API	1.48	1.22	1.0
T30	REW	1.43	1.19	0.98
T30	API	1.46	1.2	0.98
EDT	REW	1.24	1.06	1.0
EDT	API	1.32	1.08	1.02

Cuadro 1: Comparativa de parámetros acústicos (RI Sintetizada).

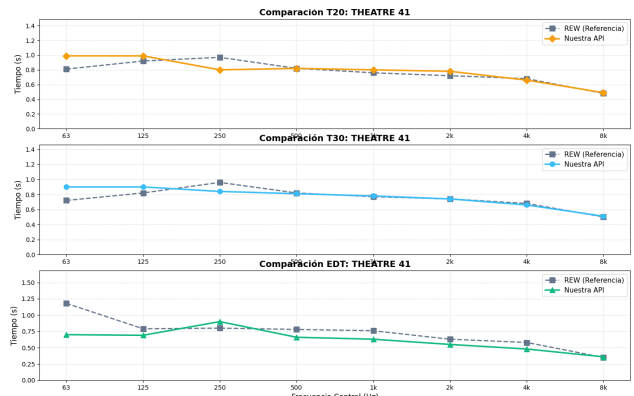


Figura 12: Comparativa paramétrica frente a REW (Theatre).

Parámetro	Software	500 Hz	1 kHz	2 kHz
T20	REW	0.82	0.76	0.72
T20	API	0.82	0.8	0.78
T30	REW	0.82	0.77	0.74
T30	API	0.81	0.78	0.74
EDT	REW	0.78	0.76	0.63
EDT	API	0.66	0.63	0.55

Cuadro 2: Parámetros en bandas clave (Theatre).

4.2. Falencias y Efecto del Método de Lundebly

A pesar del notable rendimiento general mostrado por los tests, se registraron ciertas falencias inherentes a las limitaciones algorítmicas de la API. Particularmente, en recintos extremadamente reverberantes como en el caso del Hamilton Mausoleum el algoritmo de Lundebly puede

presentar tendencias a inestabilidades (bucles infinitos) sin una calibración manual estricta. Además, en las bandas de baja frecuencia (graves) se pierde precisión debido a que la energía acústica modal de la sala no decae de manera puramente lineal, lo cual confunde la regresión.

Sin embargo, cuando el algoritmo logra estimar correctamente el ruido (Figura 14), consigue un corte limpio permitiendo ajustar curvas consistentes, a diferencia de no usarlo (Figura 13), donde el piso de ruido distorsiona completamente la integral.

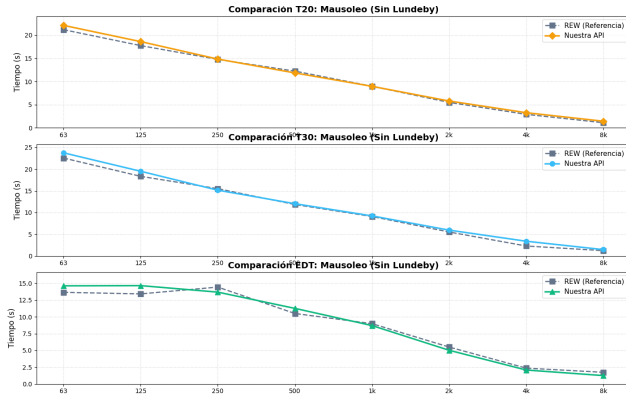


Figura 13: Ajuste de la integral sin Lundeby en recinto Mausoleo.

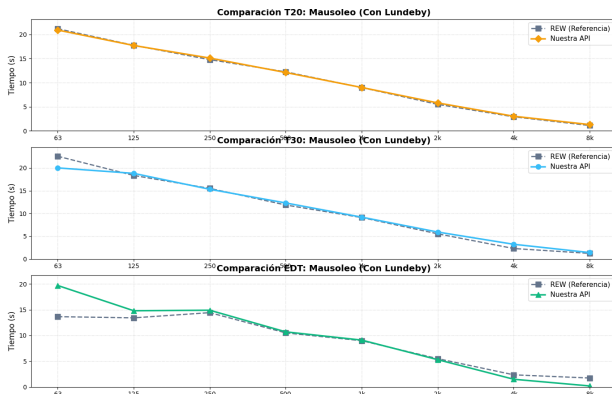


Figura 14: Ajuste de la integral utilizando Lundeby en recinto Mausoleo.

El análisis visual de las curvas de la Figura 13 evidencia que, sin corrección, la energía del ruido de fondo estacionario "levanta" la cola de la integral de Schroeder, aplanando la pendiente. Al aplicar el truncamiento de Lundeby (Figura 14), se logra aislar la reverberación pura. El Cuadro 3 cuantifica esta defensa: sin Lundeby (SL), el T20 y T30 se ven artificialmente inflados, pero al activar la corrección (CL), los valores convergen con mayor fidelidad hacia los de referencia, demostrando la necesidad crítica de este algoritmo en recintos de respuesta extrema.

Par.	Estado	Soft	500 Hz	1 kHz	2 kHz
T20	SL	REW	12.24	8.98	5.5
T20	SL	API	11.85	8.96	5.8
T20	CL	API	12.1	9.0	5.8
T30	SL	REW	11.85	9.09	5.52
T30	SL	API	12.04	9.23	5.95
T30	CL	API	12.3	9.18	5.91

Cuadro 3: Impacto del método de Lundeby (Mausoleo). SL = Sin Lundeby, CL = Con Lundeby.

5. Dificultades y Líneas Futuras

Durante el desarrollo del motor acústico, se identificaron ciertos retos técnicos y limitaciones algorítmicas, especialmente al procesar salas con alta reverberación o al intentar automatizar procesos por consola. A continuación, se detallan las falencias encontradas y el *roadmap* propuesto para resolverlas.

5.1. Retos Técnicos

- **Lundeby en salas reverberantes:** El cálculo iterativo demostró tendencia a caer en bucles infinitos en recintos con tiempos de decaimiento extremadamente largos (como el caso del Hamilton Mausoleum).
- **Precisión espectral:** Se observaron leves discrepancias de precisión en las bandas más graves (bajas frecuencias), donde la energía de la sala no decae de manera perfectamente lineal.
- **Manejo de arrays:** Surgieron dificultades al encastrar múltiples bloques de procesamiento sin fragmentar o romper la integridad de las matrices en NumPy.
- **Errores de terminal:** Fue necesario lidiar con excepciones de ejecución en tiempo real, validación de comandos y limitaciones en la obtención de gráficos de prueba locales.
- **Otros detalles:** Queda pendiente una calibración más fina de los parámetros de suavizado y la corrección del *pre-ringing* (oscilaciones previas) al aplicar el filtro inverso.

5.2. Roadmap Futuro

Para robustecer la herramienta y acercarla a un producto comercial, se proyectan las siguientes implementaciones:

- **Soporte multicanal:** Permitir la ingesta de audios estéreo y multicanal (arreglos de micrófonos).
- **Interfaz Gráfica (Frontend):** Desarrollar una interfaz de usuario interactiva (UI) para abandonar el uso estricto por consola.
- **Mejoras en Lundeby:** Calibración de ventanas dinámicas y un umbral de "Punto Seguro" adaptable (por ejemplo, al 20 %).

- **Filtros dinámicos:** Implementar un banco de filtros fraccionales dinámicos (hasta resoluciones de 1/3 de octava).
- **Distorsión y Tapering:** Añadir la medición simultánea de distorsión armónica (THD) y la resolución del problema del “clic” acústico inicial mediante *fade-in/out* (Tapering).

6. Conclusiones

El presente trabajo culminó con el desarrollo exitoso de un motor de cálculo acústico modular y escalable, construido íntegramente en Python y estructurado como una Interfaz de Programación de Aplicaciones (API). Se demostró que es posible automatizar el procesamiento digital de señales acústicas cumpliendo rigurosamente con los estándares internacionales exigidos por la norma **ISO 3382** y las especificaciones de filtros de la **IEC 61260**.

El núcleo matemático del proyecto validó su eficacia al procesar tanto respuestas al impulso artificiales como grabaciones de campo obtenidas mediante la técnica del *sine sweep* logarítmico y la deconvolución de Farina. La implementación del filtro de Butterworth bidireccional (`filtfilt`) aseguró el aislamiento de bandas de octava sin distorsión de fase, mientras que la integración inversa de Schroeder y el algoritmo iterativo de Lundeby probaron ser herramientas indispensables para trazar decaimientos puramente exponenciales y blindar el cálculo frente al ruido de fondo estacionario.

La robustez del motor quedó comprobada empíricamente a través de dos frentes: por un lado, una batería de pruebas automatizadas (*testing*) que auditó la resiliencia algorítmica frente a casos extremos; por el otro, una altísima convergencia de resultados frente a un software comercial estándar de la industria (REW). Las leves discrepancias observadas en las bandas de baja frecuencia validan la complejidad inherente a la estimación del piso de ruido en diferentes implementaciones matemáticas, reforzando la importancia de la precisión algorítmica en la ingeniería acústica.

De cara al futuro, la decisión de diseñar este motor como una API —desacoplando totalmente el procesamiento matemático (*backend*) de la interfaz gráfica (*frontend*)— sienta las bases para múltiples proyecciones. El sistema se encuentra preparado para integrar mediciones multicanal, cálculos de parámetros binaurales y análisis de distorsión armónica, e incluso para ser consumido por aplicaciones web o móviles que democratizen el acceso al análisis acústico profesional de alta precisión.

6.1. Aprendizajes y Reflexiones Finales

Más allá de los resultados técnicos, el desarrollo de la RIR-API dejó lecciones fundamentales sobre la intersección entre la acústica y el desarrollo de software. A nivel arquitectónico, la separación de lógicas complejas en *routers*, *services* y *schemas* (utilizando FastAPI y Pydantic para la validación estricta de datos) y la adopción de metodologías como *Fail-Fast*, demostraron que

una planificación sólida hace crecer las aspiraciones de cualquier proyecto.

El flujo de trabajo evidenció que el avance progresivo y la asignación de roles claros superan ampliamente el esquema tradicional de “maratón de última hora”. Herramientas como el control de versiones (trabajar en ramas de GitHub) actuaron como un verdadero seguro de vida para el equipo, previniendo regresiones en el código y facilitando la colaboración.

Finalmente, este proyecto reafirmó que no se necesita ser un experto en sistemas para poder programar; el verdadero desafío radicó en adquirir nuevos y profundos conocimientos sobre el procesamiento digital de señales (DSP) y traducirlos a algoritmos. La RIR-API es el resultado directo de la convergencia entre la teoría acústica formal y las prácticas de un producto de software moderno, demostrando que la ingeniería de audio actual no se limita a utilizar software comercial preexistente, sino a comprender, codificar y dominar los fenómenos matemáticos subyacentes.

Referencias

- [1] ISO 3382-2. *Measurement of room acoustic parameters*, 2008.
- [2] Voss, R. F., & Clarke, J. “1/f noise in music: Music from 1/f noise”. *J. Acoust. Soc. Am.* 63: 258-263, 1978.
- [3] Farina, A. “Simultaneous measurement of impulse response and distortion with a swept-sine technique”. *108th AES Convention*, 2000.
- [4] IEC 61260. *Electroacoustics - Octave-band and fractional-octave-band filters*, 1995.
- [5] Schroeder, M. R. “New method of measuring reverberation time”. *J. Acoust. Soc. Am.* 37: 409-412, 1965.
- [6] Lundeby, A., Vigran, T. E., Bietz, H., & Vorländer, M. “Uncertainties of Measurements in Room Acoustics”. *ACUSTICA* Vol. 81, 1995.